

GRAPH ALGORITHM

- A **Graph** is a non-linear data structure consisting of vertices and edges.
- The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph.
- More formally a Graph is composed of a set of **vertices(V)** and a set of **edges(E)**.
- The **graph** is denoted by **G(V, E)**.

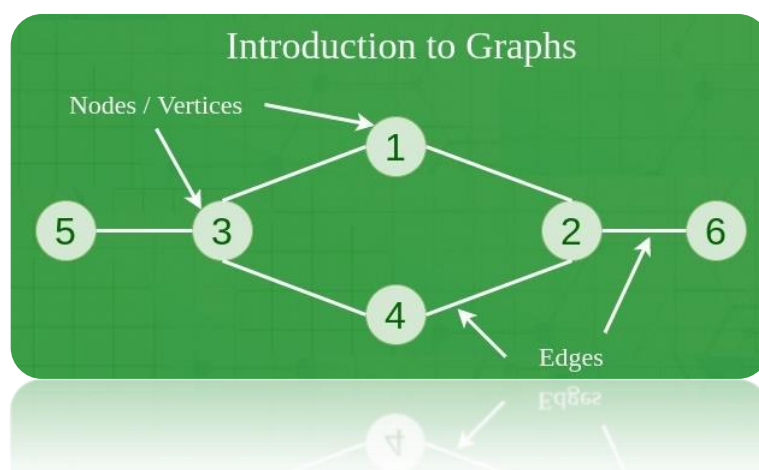
Components of a Graph

Vertices:

- ✓ Vertices are the fundamental units of the graph.
- ✓ Sometimes, vertices are also known as vertex or nodes.
- ✓ Every node/vertex can be labeled or unlabelled.

Edges:

- ✓ Edges are drawn or used to connect two nodes of the graph.
- ✓ It can be ordered pair of nodes in a directed graph.
- ✓ Edges can connect any two nodes in any possible way.
- ✓ There are no rules.
- ✓ Sometimes, edges are also known as arcs.
- ✓ Every edge can be labelled/unlabelled.



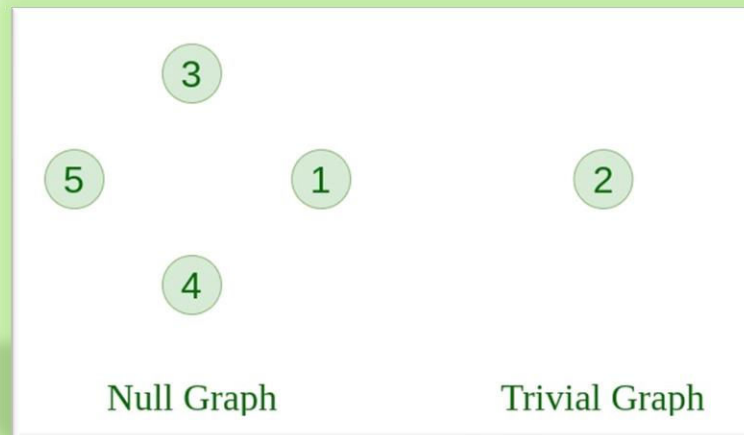
Types Of Graph

+ Null Graph

- ✓ A graph is known as a null graph if there are no edges in the graph.

+ Trivial Graph

- ✓ Graph having only a single vertex, it is also the smallest graph possible.

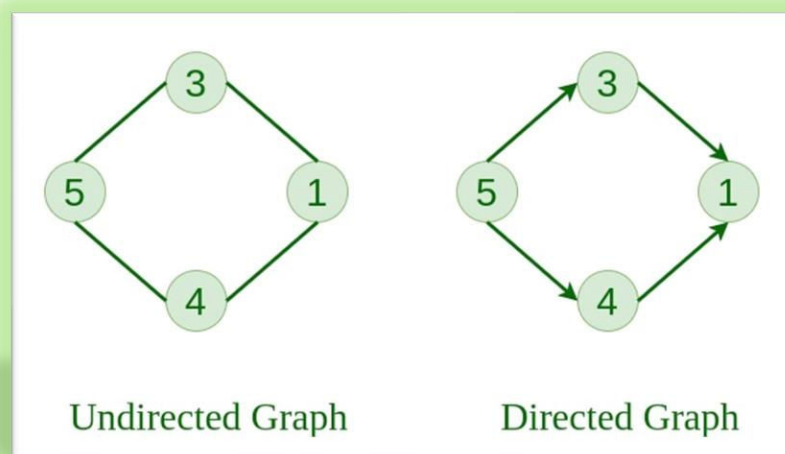


+ Undirected Graph

- ✓ A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

+ Directed Graph

- ✓ A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.

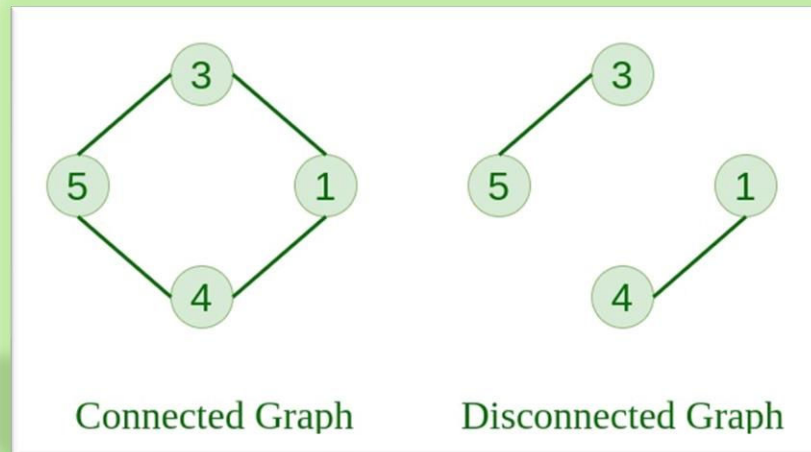


✚ Connected Graph

- ✓ The graph in which from one node we can visit any other node in the graph is known as a connected graph.

✚ Disconnected Graph

- ✓ The graph in which at least one node is not reachable from a node is known as a disconnected graph.

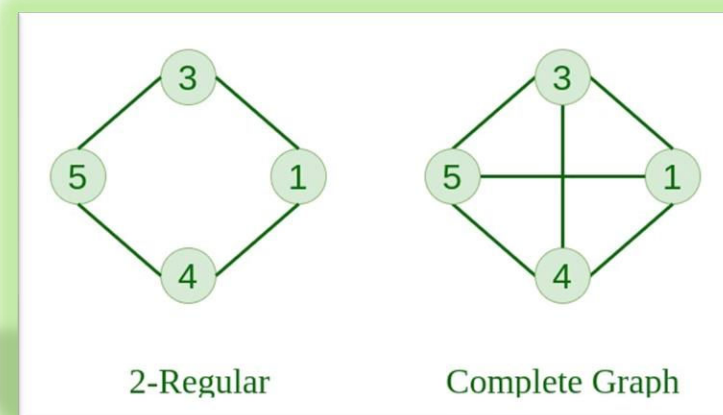


✚ Regular Graph

- ✓ The graph in which the degree of every vertex is equal to K is called K regular graph.

✚ Complete Graph

- ✓ The graph in which from each node there is an edge to each other node.

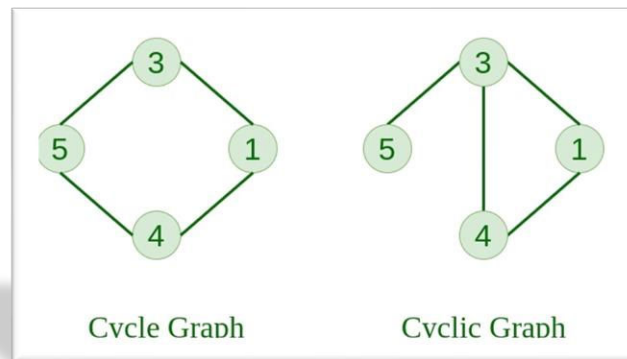


✚ Cycle Graph

- ✓ The graph in which the graph is a cycle in itself, the degree of each vertex is 2.

✚ Cyclic Graph

- ✓ A graph containing at least one cycle is known as a Cyclic graph.

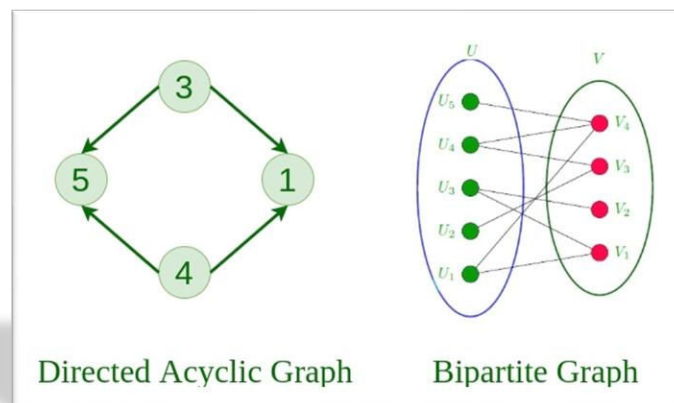


✚ Directed Acyclic Graph

- ✓ A Directed Graph that does not contain any cycle.

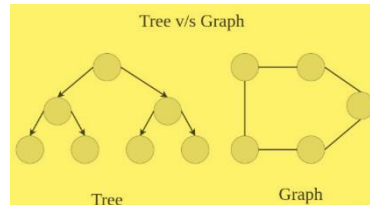
✚ Bipartite Graph

- ✓ A graph in which vertex can be divided into two sets such that vertex in each set does not contain any edge between them.



Tree v/s Graph

- ✓ Trees are the restricted types of graphs, just with some more rules.
- ✓ Every tree will always be a graph but not all graphs will be trees.
- ✓ Linked List, Trees, and Heaps all are special cases of graphs.



❖ What is Graph Algorithm ?

- A **graph algorithm** is an algorithm designed to solve problems related to graphs, which are mathematical structures consisting of vertices (nodes) and edges that connect pairs of vertices.
- Graph algorithms are used to analyze relationships and connections between entities represented by vertices, making them fundamental in various fields such as computer science, **operations research**, and **social network analysis**.

Here are some common **types** of graph algorithms:

Traversal Algorithms:

- **Breadth-First Search (BFS):** Explores a graph level by level, visiting all neighbors before moving to the next level.
- **Depth-First Search (DFS):** Explores a graph by going as deep as possible along each branch before backtracking.

Shortest Path Algorithms:

- **Dijkstra's Algorithm:** Finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative weights.
- **Bellman-Ford Algorithm:** Finds the shortest path from a source vertex to all other vertices in a weighted graph, even with negative weights.

Minimum Spanning Tree Algorithms:

- **Kruskal's Algorithm:** Finds a minimum spanning tree in an undirected graph.
- **Prim's Algorithm:** Finds a minimum spanning tree in an undirected graph.

All Pairs Shortest Path Algorithms:

- **Floyd-Warshall Algorithm:** Finds the shortest paths between all pairs of vertices in a weighted graph.
- **Topological Sorting:** Arranges the vertices of a directed acyclic graph (DAG) in a linear order such that for every directed edge (u, v) , vertex u comes before v .

Network Flow Algorithms:

- **Ford-Fulkerson Algorithm:** Solves the maximum flow problem in a flow network.

Matching Algorithms:

- **Bipartite Matching Algorithm:** Finds maximum cardinality matching in a bipartite graph.

Graph Coloring Algorithms:

- **Greedy Coloring Algorithm:** Assigns colors to vertices of a graph such that no two adjacent vertices have the same color.

Clique Finding Algorithms:

- **Bron-Kerbosch Algorithm:** Identifies all cliques (complete subgraphs) in an undirected graph.

Strongly Connected Components Algorithms:

- **Kosaraju's Algorithm:** Finds strongly connected components in a directed graph.



❖ Breadth First Search or BFS for a Graph

- **Breadth-first search (BFS)** is an algorithm for searching a tree data structure for a node that satisfies a given property.
- It starts at the tree root and explores all nodes at the present depth prior to moving on to the nodes at the next depth level.

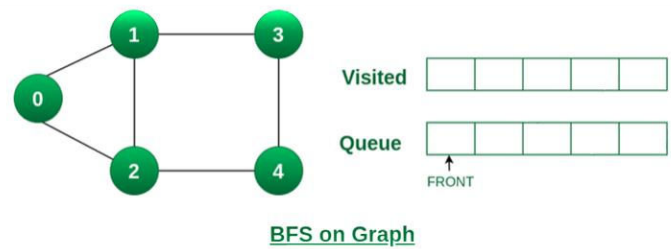
Algorithm

1. Start putting any one vertex from the graph at the back of the queue.
2. First, move the front queue item and add it to the list of the visited node.
3. Next, create nodes of the adjacent vertex of that list and add them which have not been visited yet.
4. Keep repeating steps two and three until the queue is found to be empty.

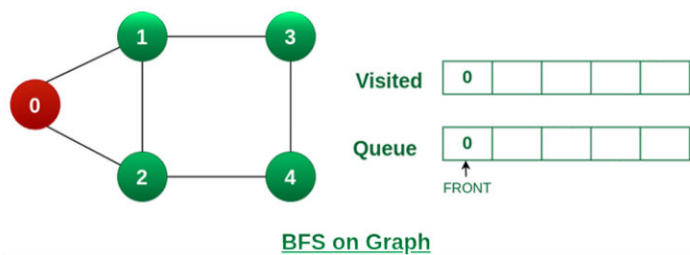
How to work DFS:

Let us understand the working of the algorithm with the help of the following **example**.

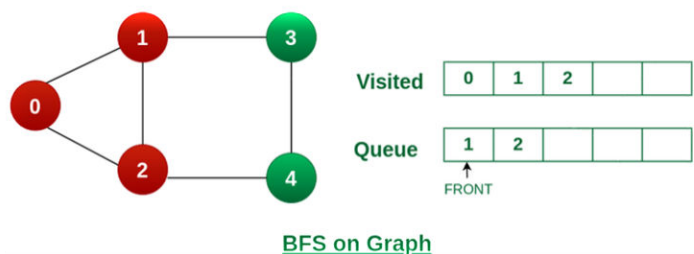
Step1: Initially queue and visited arrays are empty.



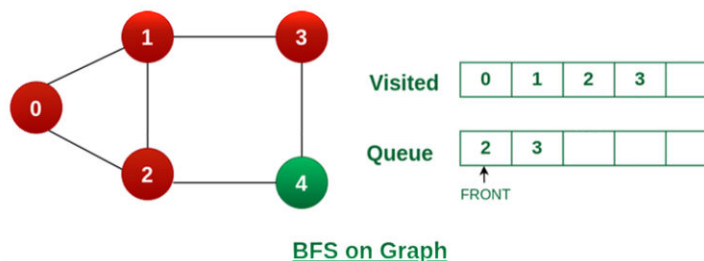
Step2: Push node **0** into queue and mark it visited.



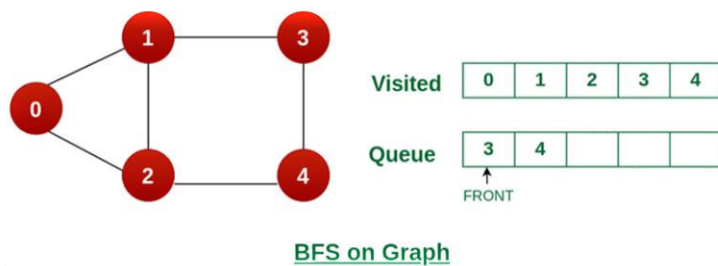
Step 3: Remove node **0** from the front of queue and visit the unvisited neighbours and push them into queue.



Step 4: Remove node **1** from the front of queue and visit the unvisited neighbours and push them into queue.

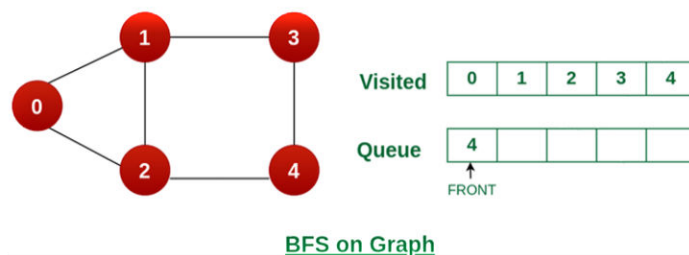


Step 5: Remove node **2** from the front of queue and visit the unvisited neighbours and push them into queue.



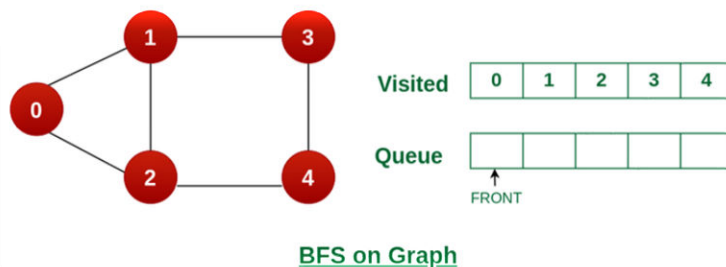
Step 6: Remove node **3** from the front of queue and visit the unvisited neighbours and push them into queue.

As we can see that every neighbours of **node 3** is visited, so move to the next node that are in the front of the queue.



Steps 7: Remove node **4** from the front of queue and visit the unvisited neighbours and push them into queue.

As we can see that every neighbours of **node 4** are visited, so move to the next node that is in the front of the queue.



Now, Queue becomes empty, So, terminate these process of iteration.

Implementations of BFS:

```

/ C++ code to print BFS traversal from a given
#include <bits/stdc++.h>
using namespace std;
// This class represents a directed graph using
// adjacency list representation
class Graph {
// No. of vertices
int V;
// Pointer to an array containing adjacency lists
vector<list<int> > adj;
public:
// Constructor
Graph(int V);
// Function to add an edge to graph
void addEdge(int v, int w);
// Prints BFS traversal from a given source s
void BFS(int s);
};
Graph::Graph(int V)
{
this->V = V;
adj.resize(V);
}
void Graph::addEdge(int v, int w)
{
// Add w to v's list.
adj[v].push_back(w);
}

```

```

void Graph::BFS(int s)
{
    // Mark all the vertices as not visited
    vector<bool> visited;
    visited.resize(V, false);

    // Create a queue for BFS
    list<int> queue;

    // Mark the current node as visited and enqueue it
    visited[s] = true;
    queue.push_back(s);
    while (!queue.empty()) {
        // Dequeue a vertex from queue and print it
        s = queue.front();
        cout << s << " ";
        queue.pop_front();
        // Get all adjacent vertices of the dequeued
        // vertex s.
        // If an adjacent has not been visited,
        // then mark it visited and enqueue it
        for (auto adjacent : adj[s]) {
            if (!visited[adjacent]) {
                visited[adjacent] = true;
                queue.push_back(adjacent);
            }
        }
    }
    // Driver code
    int main()
    {
        // Create a graph given in the above diagram
        Graph g(4);
    }
}

```

```

g.addEdge(0, 1);
g.addEdge(0, 2);
g.addEdge(1, 2);
g.addEdge(2, 0);
g.addEdge(2, 3);
g.addEdge(3, 3);

cout << "Following is Breadth First Traversal "
<< "(starting from vertex 2) \n";
g.BFS(2);
return 0;
}

```

Time Complexity: $O(V+E)$, where V is the number of nodes and E is the number of edges.

Auxiliary Space: $O(V)$.

❖ DFS (Depth First Search) algorithm

- It is a **recursive algorithm** to search all the vertices of a tree data structure or a graph.
- The **depth-first search (DFS) algorithm** starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.
- Because of the recursive nature, stack data structure can be used to implement the DFS algorithm.
- The process of implementing the DFS is similar to the BFS algorithm.

The step by step process to **implement the DFS** traversal is given as follows:

- First, **create a stack** with the total number of vertices in the graph.
- Now, **choose any vertex** as the **starting point of traversal**, and **push** that vertex into the stack.
- After that, push a non-visited vertex (adjacent to the vertex on the top of the stack) to the top of the stack.
- Now, **repeat steps 3 and 4** until no vertices are left to visit from the vertex on the stack's top.
- If no vertex is left, go back and pop a vertex from the stack.
- Repeat **steps 2, 3, and 4** until the stack is empty.

Applications of DFS algorithm:

- ✓ The applications of using the DFS algorithm are given as follows -
- ✓ DFS algorithm can be used to implement the topological sorting.
- ✓ It can be used to find the paths between two vertices.
- ✓ It can also be used to detect cycles in the graph.
- ✓ DFS algorithm is also used for one solution puzzles.
- ✓ DFS is used to determine if a graph is bipartite or not.

Algorithm:

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Push the starting node A on the stack and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until STACK is empty

Step 4: Pop the top node N. Process it and set its STATUS = 3 (processed state)

Step 5: Push on the stack all the neighbors of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)

[END OF LOOP]

Step 6: EXIT

Pseudocode:

```

1. DFS(G,v) ( v is the vertex where the search starts )
2. Stack S := {}; ( start with an empty stack )
3. for each vertex u, set visited[u] := false;
4. push S, v;
5. while (S is not empty) do
6. u := pop S;
7. if (not visited[u]) then
8. visited[u] := true;
10. for each unvisited neighbour w of u
11. push S, w;
12. end if
13. end while
14. END DFS()

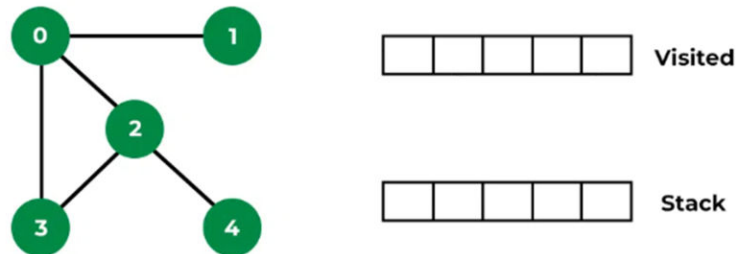
```

How does DFS work?

- Depth-first search is an algorithm for traversing or searching tree or graph data structures.
- The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

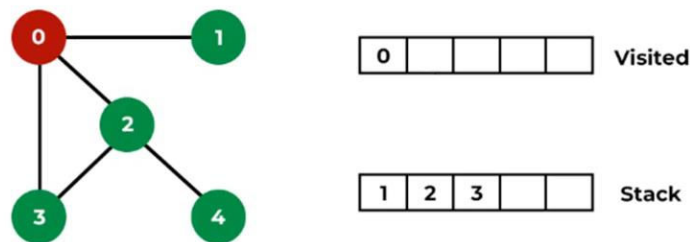
Let us understand the **working** of **Depth First Search** with the help of the following illustration:

Step1: Initially stack and visited arrays are empty.



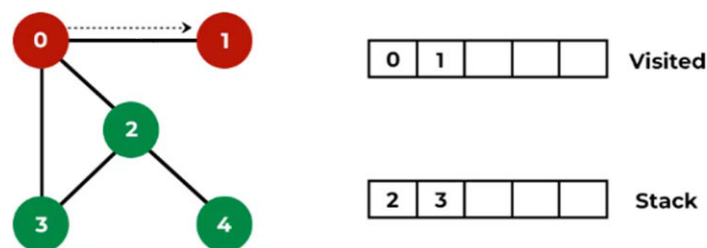
DFS on Graph

Step 2: Visit 0 and put its adjacent nodes which are not visited yet into the stack.



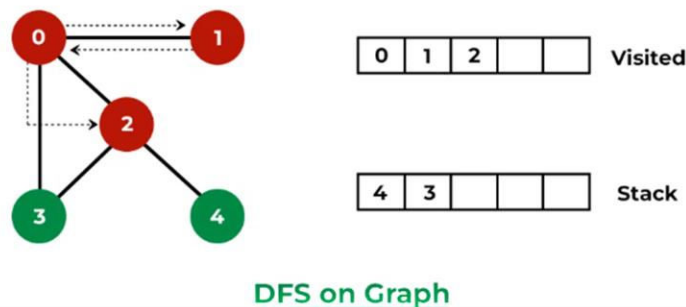
DFS on Graph

Step 3: Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.

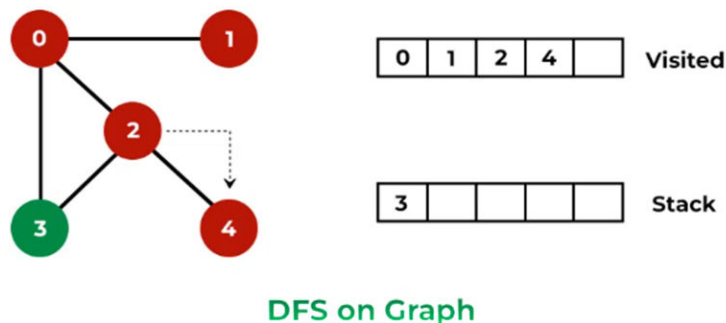


DFS on Graph

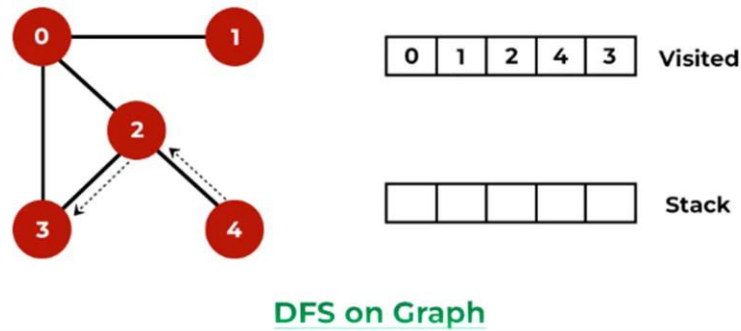
Step 4: Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.



Step 5: Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Step 6: Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Now, Stack becomes empty, which means we have visited all the nodes and our DFS traversal **ends**.

Implementation of DFS:

```
// C++ program to print DFS traversal from
// a given vertex in a given graph
#include <bits/stdc++.h>
using namespace std;
// Graph class represents a directed graph
// using adjacency list representation
class Graph {
public:
    map<int, bool> visited;
    map<int, list<int> > adj;
    // Function to add an edge to graph
    void addEdge(int v, int w);
    // DFS traversal of the vertices
    // reachable from v
    void DFS(int v);
};
```

```

void Graph::addEdge(int v, int w)
{
    // Add w to v's list.
    adj[v].push_back(w);
}

void Graph::DFS(int v)
{
    // Mark the current node as visited and
    // print it
    visited[v] = true;
    cout << v << " ";

    // Recur for all the vertices adjacent
    // Driver code
}

int main()
{
    // Create a graph given in the above diagram
    Graph g;
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    cout << "Following is Depth First Traversal"
           " (starting from vertex 2) \n";
}

```

```
// Function call
g.DFS(2);

return 0;
}

// to this vertex
list<int>::iterator i;
for (i = adj[v].begin(); i != adj[v].end(); ++i)
    if (!visited[*i])
        DFS(*i);
}
```

OUTPUT:

```
Following is Depth First Traversal (starting from vertex 2)
2 0 1 3
```

✚ Time complexity: $O(V + E)$

✚ Auxiliary Space: $O(V + E)$